기초 강의 · 자율주행 자산운용 준비

AI·멀티에이전트 구현의 기초

자율주행 자산운용 시스템을 Claude Code로 만들기 위한 개념 강의

모델에서 출발해 에이전트, 멀티에이전트, 하네스 구현까지 차례로 쌓는다

자산운용 지식이 아니라 에이전트 시스템을 설계하고 만드는 데 필요한 기본 개념을 다룬다.
금융 예시는 이해를 돕는 정도로만 쓴다.

에이전트 시스템은 세 층을 쌓아 만든다

모델 → 에이전트 → 멀티에이전트 → 하네스 순서로 읽는다

1

에이전트는 모델에 도구와 반복 루프를 붙인 것이다

모델은 다음 말을 예측할 뿐이다. 도구를 쓰고 결과를 확인하며 반복할 때 비로소 일을 끝낸다.

2

멀티에이전트의 가치는 역할 분리와 상호 검증에 있다

에이전트를 여럿 두는 이유는 속도보다 신뢰다. 서로 다른 관점이 서로의 오류를 잡아야 한다.

3

Claude Code는 이 구조를 파일로 구현하는 하네스다

규칙은 CLAUDE.md에, 역할은 서브에이전트에, 절차는 스킬에, 금지선은 훅에 적는다.

목표: 사람이 최종 결정하는 L3 자율 운용 체계를 직접 설계할 수 있게 된다

네 부분으로 기초부터 구현까지 올라간다

각 부는 앞 부의 개념을 재료로 쓴다

부	주제	핵심 질문
1부	생성형 AI의 기본 단위	모델은 무엇을 하고 무엇을 못 하는가?
2부	에이전트의 작동 원리	모델이 어떻게 스스로 일을 끝내는가?
3부	멀티에이전트의 설계	여러 에이전트를 왜, 어떻게 엮는가?
4부	Claude Code로 구현하기	이 구조를 실제 파일로 어떻게 만드는가?

이 강의는 Self-Driving Portfolio 덱(Ang 외, 2026)과 팀 프로젝트를 위한 선행 학습이다

01

1부 · 기본 단위

생성형 AI의 기본 단위

언어 모델이 하는 일과 그 한계를 먼저 정확히 이해한다

언어 모델은 다음 토큰을 예측하는 모델이다

LLM(Large Language Model)의 정의부터 확인한다

$$\underbrace{x_t}_{\text{다음 토큰}} \sim p_\theta\Big(\,\cdot \;\Big|\; \underbrace{x_1, x_2, \ldots, x_{t-1}}_{\text{지금까지의 문맥}}\Big)$$

할 수 있는 것

- 문맥을 읽고 그럴듯한 다음 말을 이어 쓴다
- 요약, 번역, 코드 작성처럼 언어로 된 일을 잘한다
- 지시문만 바꿔도 행동이 달라진다

알아야 할 한계

- 사실이 아니어도 그럴듯하게 쓸 수 있다(환각)
- 대화가 끝나면 아무것도 기억하지 않는다
- 계산과 최신 정보는 도구를 써야 정확해진다

모델은 컨텍스트 안에 있는 것만 본다

컨텍스트 윈도우(context window)는 모델의 작업 메모리다

컨텍스트에 넣지 않은 정보는 모델에게 존재하지 않는다

토큰

모델이 읽고 쓰는 글자 조각 단위다.
길이와 비용을 재는 기준이 된다.

윈도우

한 번에 볼 수 있는 토큰의 상한이다.
넘치면 앞부분을 요약해 압축한다.

컨텍스트 엔지니어링

무엇을 넣고 무엇을 뺄지 설계하는
일이다. 에이전트 성능을 좌우한다.

에이전트 설계의 절반은 어떤 정보를 언제 컨텍스트에 넣을지 정하는 일이다

컨텍스트는 네 가지 재료로 채워진다

에이전트가 한 번 판단할 때 들어가는 재료를 나눠 본다

재료	무엇인가	Claude Code에서는
시스템 프롬프트	역할과 행동 원칙을 정하는 지시문	CLAUDE.md, 서브에이전트 본문
사용자 지시	이번에 할 과제	입력창에 쓰는 프롬프트
도구 결과	파일, 검색, 계산이 돌려준 값	Read, Bash, MCP 도구의 결과
기억	이전 작업에서 남긴 기록	메모리 파일, runs/ 실행 기록

같은 모델이라도 넣어 준 재료가 다르면 전혀 다른 답을 낸다

모델은 기억이 없으므로 기억은 파일에 둔다

상태가 없다(stateless)는 뜻과 그 대응을 정리한다

상태가 없다는 것

- 새 세션은 늘 백지에서 시작한다
- 어제 내린 결정의 이유를 모델은 모른다
- 긴 대화는 압축되면서 세부가 사라진다

그래서 파일에 적는다

- 규칙과 용어는 CLAUDE.md에 둔다
- 결정과 근거는 실행 기록(runs/)에 남긴다
- 다음 세션은 이 파일을 읽고 이어 간다

인수인계서 없는 새 담당자처럼, 기록 없는 에이전트는 매번 처음부터 시작한다

02

2부 · 에이전트

에이전트의 작동 원리

모델이 도구를 쓰며 스스로 일을 끝내는 구조를 본다

에이전트는 목표를 받아 끝까지 반복한다

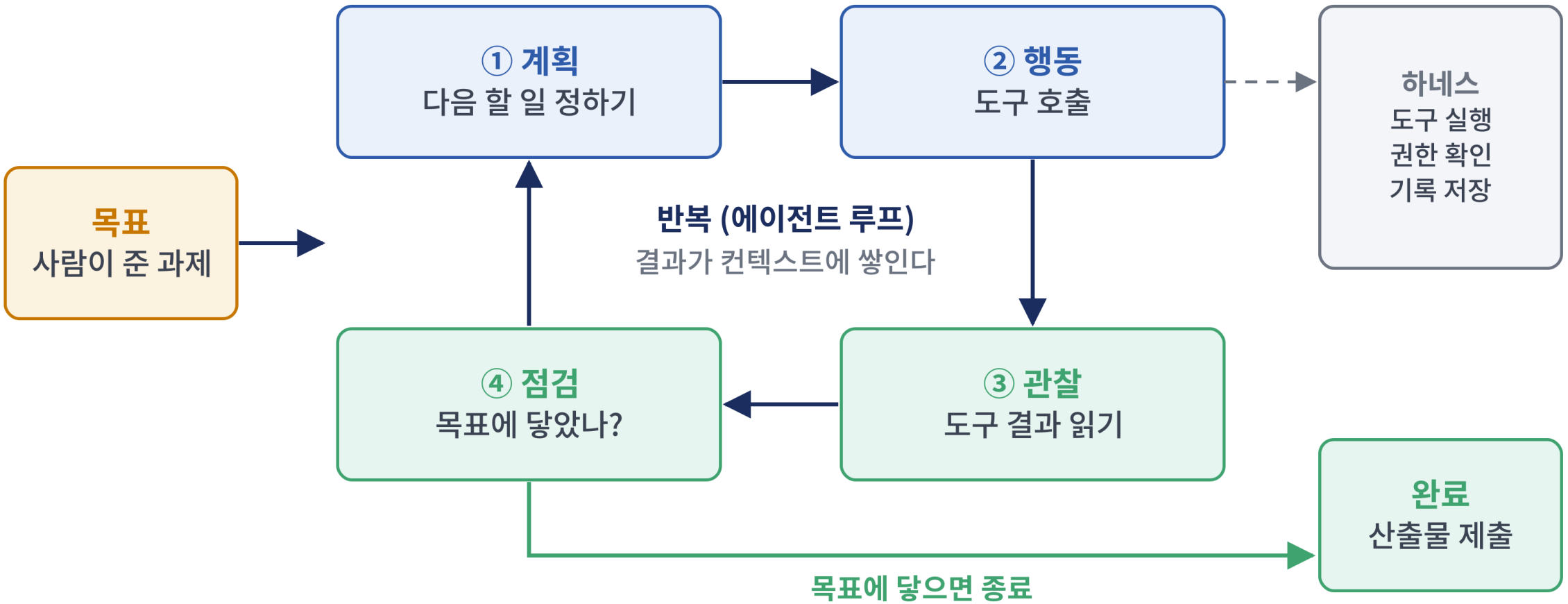
챗봇과 에이전트의 차이를 한 문장으로 정의한다

에이전트 = 모델 + 도구 + 반복 루프 + 기억

구분	챗봇	에이전트
다음 단계는 누가 정하나	사람이 매번 묻는다	모델이 스스로 고른다
도구	쓰지 않거나 한 번 쓴다	필요할 때마다 반복해서 쓴다
언제 끝나나	답을 한 번 내면 끝난다	목표 점검을 통과하면 끝난다
실패하면	틀린 답 하나가 남는다	틀린 행동이 연쇄로 쌓인다

에이전트는 네 단계를 돌며 목표에 다가간다

도구 실행과 권한 확인은 모델이 아니라 하네스가 맡는다



에이전트 루프는 한 줄의 점화식으로 적힌다

정책 π(policy)가 컨텍스트를 보고 다음 행동을 고른다

$$\underbrace{a_t}_{\text{행동(도구 호출)}} = \pi_\theta\Big(\underbrace{c_t}_{\text{현재 컨텍스트}}\Big), \qquad \underbrace{o_t}_{\text{관찰 결과}} = \mathrm{Tool}(a_t), \qquad \underbrace{c_{t+1} = c_t \oplus (a_t,\, o_t)}_{\text{결과를 컨텍스트에 덧붙임}}$$

식 읽는 법

- π는 모델 자체다. 컨텍스트를 받아 다음 행동을 낸다
- 행동의 결과가 컨텍스트에 덧붙어 다음 판단의 재료가 된다

설계에 주는 함의

- 컨텍스트가 길어질수록 비용이 늘고 초점이 흐려진다
- 그래서 긴 일은 나눠서 다른 에이전트에 맡긴다

에이전트는 네 가지 부품으로 이루어진다

어느 부품이 약해도 에이전트 전체가 흔들린다

역할 지시

System prompt

무엇을 맡고 무엇을 하지 않는지 적은 지시문

예) 리스크만 본다

도구

Tools

파일 읽기, 코드 실행, 검색처럼 바깥세상을 건드리는 수단

예) 변동성 계산

기억

Memory

세션을 넘어 남겨야 할 규칙과 기록

예) 지난 회의 결론

점검

Evaluation

결과가 목표와 규칙을 만족하는지 확인하는 장치

예) 한도 위반 검사

에이전트 설계란 이 네 부품을 역할마다 다르게 조합하는 일이다

모델은 도구를 직접 실행하지 않고 요청만 낸다

도구 호출(tool use)이 실제로 오가는 모습을 본다

[출력] 모델이 낸 도구 호출 요청

```
{
  "tool": "run_python",
  "input": {
    "script": "src/vol.py",
    "args": ["--asset", "KOSPI200"]
  }
}
```

[출력] 하네스가 실행하고 돌려준 결과

```
{
  "ok": true,
  "annual_vol": 0.183,
  "window": "252d"
}
```

왜 이렇게 나누는가

- 모델은 숫자 계산을 틀릴 수 있으므로 계산은 코드가 맡는다
- 하네스가 실행 전에 권한을 확인할 수 있어 안전하다

자율성은 사람이 개입하는 지점으로 구분한다

자율주행차의 단계 구분을 에이전트에 옮겨 본다(이 강의의 구분)

단계	누가 결정하나	예시
L0	사람이 모두 한다. AI는 질문에 답만 한다	답을 복사해 쓴다
L1	AI가 도구를 쓰되 사람이 매번 승인한다	명령마다 허락한다
L2	정해진 절차를 AI가 자동으로 돈다	매일 같은 리포트
L3	목표를 맡기고 사람은 관문에서 승인한다	초안은 AI, 결재는 CIO
L4	사람은 감독만 하고 예외 때만 개입한다	한도 안에서 자동 실행

이 강의가 겨냥하는 수준은 L3다. 에이전트가 끝까지 일하고 사람이 결재한다

정해진 흐름과 자율 판단 사이에 다섯 패턴이 있다

Anthropic(2024) 'Building Effective Agents'의 분류를 따른다

패턴	작동 방식	자산운용 예
프롬프트 체이닝	앞 단계 출력을 다음 단계 입력으로 넘긴다	뉴스 요약 → 시사점 → 보고서
라우팅	입력을 분류해 맞는 처리로 보낸다	질의를 주식 또는 채권 담당으로 보낸다
병렬화	일을 나눠 동시에 하고 합친다	자산군별 시나리오를 동시에 계산한다
오케스트레이터-워커	리드가 과제를 쪼개 워커에게 맡긴다	리서치 주제를 하위 질문으로 나눈다
평가자-최적화	생성자와 평가자가 개선을 반복한다	제안서 작성과 비평을 되풀이한다

위쪽 셋은 흐름을 사람이 정하는 워크플로이고, 아래로 갈수록 모델이 흐름을 정한다

03

3부 · 멀티에이전트

멀티에이전트의 설계

여러 에이전트를 왜, 어떻게 엮는지 본다

에이전트를 나누는 이유는 신뢰를 얻기 위해서다

한 에이전트에 모든 일을 맡길 때 생기는 문제에서 출발한다

전문화

Specialization

역할 지시를 좁히면 각자 더 정확해진다. 리서치와 리스크를 한 지시문에 섞지 않는다.

컨텍스트 격리

Isolation

각 에이전트가 자기 컨텍스트만 쓴다. 긴 검색 결과가 다른 판단을 흐리지 않는다.

상호 검증

Cross-check

만드는 쪽과 검토하는 쪽을 나눈다. 자기 오류는 자기가 잘 보지 못한다.

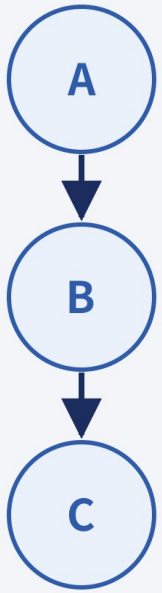
투자위원회를 운용역과 따로 두는 이유와 같다. 이른바 4-eyes 원칙이다

멀티에이전트는 네 가지 기본 구조로 엮인다

실제 시스템은 이 넷을 섞어 쓴다. 리드는 일을 나누는 에이전트다

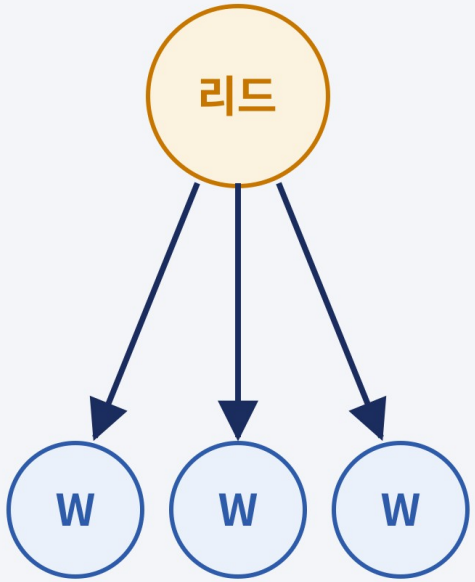
파이프라인

순서대로 넘긴다



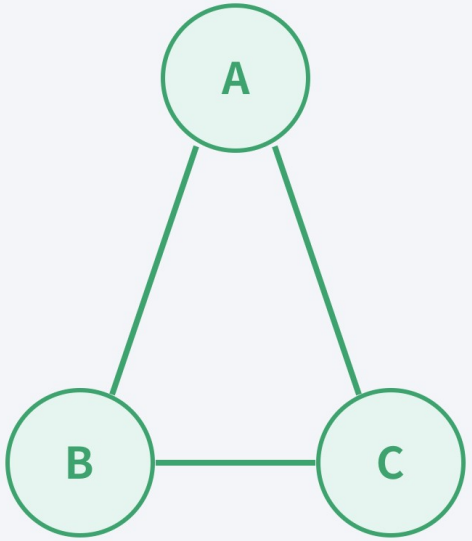
오케스트레이터-워커

나눠 맡기고 모은다



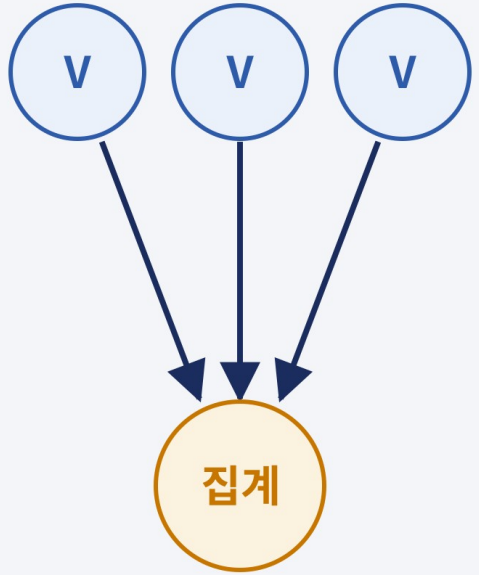
토론·상호 검토

서로 반박한다



투표·앙상블

결과를 집계한다



리드-서브에이전트 구조는 리서치에서 효과가 컸다

사례: Anthropic 멀티에이전트 리서치 시스템(2025)

+90.2%

단일 에이전트(Opus 4) 대비 내부 리서치 평가 성능. 리드는 Opus 4, 서브는 Sonnet 4

약 15배

일반 채팅 대비 멀티에이전트의 토큰 사용량. 단일 에이전트는 약 4배

80%

BrowseComp 성능 차이 가운데 토큰 사용량 하나로 설명되는 몫

잘 맞는 일

- 넓게 찾아 모아야 하는 리서치
- 서로 독립적으로 나눌 수 있는 일

잘 안 맞는 일

- 모두가 같은 맥락을 공유해야 하는 일
- 단계끼리 얽힌 일, 가치가 낮은 일

에이전트가 늘면 비용도 거의 비례해 는다

에이전트마다 자기 컨텍스트를 따로 읽기 때문이다

$$\text{총비용} = \sum_{i \in \text{에이전트}} \left(T_i^{\text{입력}} \cdot c^{\text{입력}} + T_i^{\text{출력}} \cdot c^{\text{출력}} \right)$$

식 읽는 법

- T는 토큰 수, c는 토큰당 단가다
- 입력 토큰은 에이전트마다 되풀이해 읽힌다
- 출력 토큰은 대개 입력보다 단가가 높다

설계 원칙

- 가치가 큰 판단에만 에이전트를 늘린다
- 반복 계산은 코드로, 판단만 모델에 맡긴다
- 서브에이전트는 요약만 돌려주게 한다

다수결은 판단이 서로 독립일 때만 강해진다

콩도르세 배심원 정리(Condorcet jury theorem)를 쓴다

$$P(\text{다수결이 맞을 확률}) = \sum_{k>n/2} \binom{n}{k} p^k (1-p)^{n-k}$$

기호와 숫자

- n은 에이전트 수, p는 하나가 맞힐 확률이다
- p = 0.6, n = 5이면 다수결 정답률은 0.683이다
- 같은 조건에서 n = 21이면 0.826까지 오른다

단서

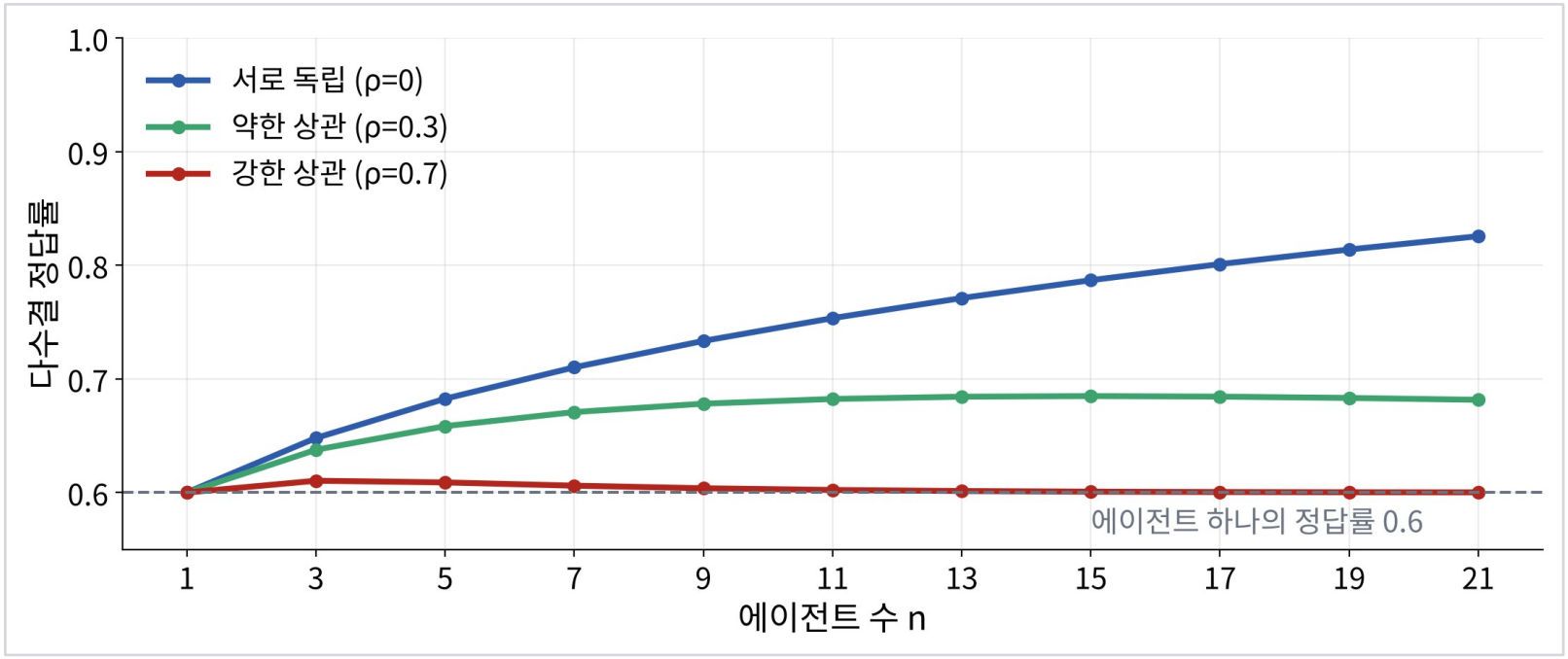
- 같은 모델과 같은 지시면 틀릴 때 함께 틀린다
- 그러면 숫자를 늘려도 정확도가 거의 오르지 않는다

상관이 강하면 에이전트를 늘려도 소용없다

ρ는 각 에이전트가 공통 판단을 그대로 따를 확률이다(개별 정답률 0.6)

읽는 법

- 파랑: 서로 독립
- 초록: 약한 상관
- 빨강: 강한 상관
- 다양성이 정확도다



순위를 모을 때는 보르다 점수를 쓴다

에이전트 n명이 후보 m개에 매긴 순위를 하나로 합친다

$$\text{보르다 점수}_j = \sum_{i=1}^{n} \left(m - r_{ij} \right) \qquad (r_{ij}: \text{ 에이전트 } i\text{가 후보 } j\text{에 매긴 순위})$$

후보	에이전트 1	에이전트 2	에이전트 3	보르다 점수 (m = 3)
전략 A	1위	2위	1위	2 + 1 + 2 = 5
전략 B	2위	1위	3위	1 + 2 + 0 = 3
전략 C	3위	3위	2위	0 + 0 + 1 = 1

A가 1위다. Self-Driving Portfolio 논문도 전략끼리 서로 평가한 순위를 이렇게 모은다

멀티에이전트는 고유한 실패 방식을 가진다

흔한 실패와 대응책을 짝지어 기억한다

실패	무슨 일이 생기나	대응
오류 전파	앞 에이전트의 환각을 뒤가 사실로 받는다	근거 파일과 출처를 함께 넘긴다
동조	비평가가 생성자 의견에 쉽게 동의한다	반대 의견을 의무로 적게 한다
무한 루프	점검을 통과하지 못해 같은 일을 되풀이한다	최대 반복 횟수를 정한다
역할 이탈	리스크 담당이 매매 의견까지 낸다	역할에 맞게 도구와 권한을 막는다
비용 폭증	에이전트가 서로를 계속 부른다	호출 깊이와 예산 상한을 둔다

대응책 대부분은 4부에서 훅과 권한 설정으로 구현한다

에이전트끼리는 정해진 형식의 파일로 일을 넘긴다

핸드오프(handoff) 형식을 미리 정해 두면 오해가 줄어든다

[출력] 리스크 에이전트가 남긴 risk_report.json

```
{
  "run_id": "2026-10-26_01",
  "agent": "risk",
  "inputs": ["data/prices.csv"],
  "vol_annual": 0.112,
  "limit": 0.10,
  "breach": true,
  "evidence": "src/vol.py 실행 결과",
  "next": "pm 에이전트에 축소안 요청"
}
```

형식을 정하는 이유

- 받는 쪽이 필드 이름으로 바로 읽는다
- 근거 파일 경로가 있어 검증할 수 있다
- run_id로 실행 전체를 추적한다

04

4부 · Claude Code 구현

Claude Code로 구현하기

앞에서 배운 개념마다 대응하는 Claude Code 기능을 짝짓는다

Claude Code는 모델을 감싸는 하네스다

하네스(harness)는 모델이 도구를 쓰고 기억하도록 돕는 실행 틀이다

모델이 두뇌라면, 하네스는 손발이자 규칙집이자 기록장이다

루프 실행

Loop

에이전트 루프를 돌리고
결과를 컨텍스트에 붙인다

도구 제공

Tools

파일 읽기와 쓰기, 셸 명령,
웹 검색, MCP 도구를
준다

권한 관리

Permissions

위험한 행동 앞에서
사람에게 묻거나 막는다

기억과 기록

Memory

CLAUDE.md를 읽고
세션 기록을 남긴다

개념마다 대응하는 Claude Code 기능이 있다

앞에서 배운 개념을 실제 파일과 명령으로 옮긴다

개념	Claude Code 기능	위치
공통 규칙	CLAUDE.md	프로젝트 루트
역할별 에이전트	서브에이전트	.claude/agents/이름.md
반복 절차 지식	스킬	.claude/skills/이름/SKILL.md
외부 데이터 연결	MCP 서버	.mcp.json
결정적 가드레일	훅	.claude/settings.json
사람 관문	권한 모드, 플랜 모드	실행 중 승인 화면
무인 반복 실행	헤드리스 실행 (claude -p)	스케줄러, CI

프로젝트 폴더 하나가 곧 시스템 설계도다

Claude Code는 이 폴더 구조를 읽고 그대로 동작한다

```
sdam/
├── CLAUDE.md ---------- 항상 읽는 프로젝트 규칙
├── .claude/
│   ├── agents/ ---------- 서브에이전트 정의 (역할별 .md)
│   ├── skills/ ---------- 스킬 (절차 지식, SKILL.md)
│   └── settings.json ---------- 권한 · 훅(가드레일)
├── .mcp.json ---------- 외부 도구 · 데이터 연결
├── data/ ---------- 입력 데이터 (읽기 전용)
├── src/ ---------- 계산 코드 (Python)
└── runs/2026-10-26_01/ ---------- 실행 기록 · 산출물 · 로그
```

CLAUDE.md는 세션마다 먼저 읽히는 규칙집이다

프로젝트 전체에 늘 적용할 사실과 규칙만 적는다

[출력] Claude가 생성 · CLAUDE.md

```
# 자율주행 운용 시스템 공통 규칙
## 목적
- 투자정책서(IPS) 안에서 운용안 초안을 만든다
- 최종 결정은 사람 CIO가 한다
## 원칙
- 숫자 계산은 src/ 의 코드로만 한다
- data/ 원본은 절대 고치지 않는다
- 산출물은 runs/<run_id>/ 에 저장한다
## 용어
- 한도: IPS 4.2절의 위험 한도
```

쓰는 요령

- 짧게 쓴다. 매번 전부 읽히므로 길면 비용이 든다
- 긴 절차는 스킬로 뺀다
- 하지 말 것까지 분명히 적는다

서브에이전트는 독립 컨텍스트를 가진 담당자다

파일 하나가 에이전트 하나다. 예: .claude/agents/risk.md

[출력] Claude가 생성 · .claude/agents/risk.md

```
---
name: risk
description: 포트폴리오 위험을 계산하고
  IPS 한도 위반 여부를 판정한다.
  운용안이 나오면 반드시 사용한다.
tools: Read, Write, Bash
model: sonnet
---
너는 리스크 담당이다. 매매 의견은 내지 않는다.
1. src/risk.py 로 변동성을 계산한다
2. 결과를 risk_report.json 에 쓴다
3. 한도 위반이면 breach: true 로 적는다
```

핵심 필드

- description: 언제 부를지 알려 준다
- tools: 도구를 좁혀 역할 이탈을 막는다
- 본문: 이 에이전트의 시스템 프롬프트가 된다
- 결과는 요약만 메인 대화로 돌아온다

스킬은 필요할 때만 펼쳐 보는 절차 매뉴얼이다

설명은 늘 보이고, 본문은 쓸 때만 컨텍스트에 들어온다

[출력] Claude가 생성 · .claude/skills/ips-check/SKILL.md

```
---
name: ips-check
description: 운용안이 투자정책서(IPS) 한도를
  지키는지 점검할 때 쓴다.
---
## 절차
1. ips.md 에서 자산군별 상한과 하한을 읽는다
2. 운용안 비중과 한 줄씩 대조한다
3. 위반 항목과 초과 폭을 표로 정리한다
4. 위반이 없으면 "통과"라고만 적는다
```

다른 장치와의 차이

- CLAUDE.md는 늘 전부 읽힌다
- 스킬은 필요할 때만 본문을 읽는다
- 서브에이전트는 별도 컨텍스트에서 일한다
- /ips-check 로 직접 부를 수도 있다

훅은 모델이 잊어도 반드시 실행되는 규칙이다

훅(hook)은 정해진 시점에 자동으로 도는 셸 명령이다

시점	언제 실행되나
PreToolUse	도구 실행 직전. 막을 수 있다
PostToolUse	도구 실행이 성공한 직후
Stop	Claude가 응답을 마칠 때
SessionStart	세션이 시작되거나 재개될 때
SubagentStop	서브에이전트가 끝날 때

[출력] Claude가 생성 · .claude/settings.json

```
{
 "hooks": {
  "PreToolUse": [{
   "matcher": "Edit|Write",
   "hooks": [{
    "type": "command",
    "command": "python3 guards/no_raw_edit.py"
   }]
  }]
 }
}
```

검사 스크립트가 종료 코드 2를 내면 그 행동은 실행되지 않는다

MCP는 외부 데이터와 도구를 잇는 표준 규격이다

MCP(Model Context Protocol)는 AI용 공용 연결 단자다

[입력] 명령어 · 터미널에 입력 (주소와 이름은 예시)

```
claude mcp add --transport http market-data https://example.com/mcp
```

무엇이 좋아지나

- 데이터 소스마다 코드를 새로 짤 필요가 없다
- 같은 서버를 여러 에이전트가 함께 쓴다
- 서브에이전트별로 쓸 서버를 제한할 수 있다

주의할 점

- 연결한 서버가 할 수 있는 일을 먼저 확인한다
- 쓰기 권한이 있는 서버는 훅과 권한으로 막는다

목적에 따라 서브에이전트와 에이전트 팀을 고른다

에이전트 팀(agent teams)은 실험 기능이며 기본으로 꺼져 있다

구분	서브에이전트	에이전트 팀
컨텍스트	각자 독립, 결과만 호출자에게 간다	각자 독립, 완전히 별도 세션이다
소통	호출자에게 결과를 돌려준다	팀원끼리 직접 메시지를 주고받는다
조율	메인 에이전트가 모두 관리한다	공유 작업 목록으로 나눠 맡는다
잘 맞는 일	결과만 중요한 집중 과제	토론과 협업이 필요한 복잡한 일
토큰 비용	낮다	높다

표준 설계는 서브에이전트로 시작하고, 토론이 필요한 단계에서만 팀을 시험한다

AI가 한 말의 책임은 운영자가 진다

사례: 에어캐나다 챗봇 판결(캐나다 BC 분쟁해결심판소, 2024)

회사는 챗봇이 별개의 주체라고 주장했지만 받아들여지지 않았다

무슨 일이 있었나

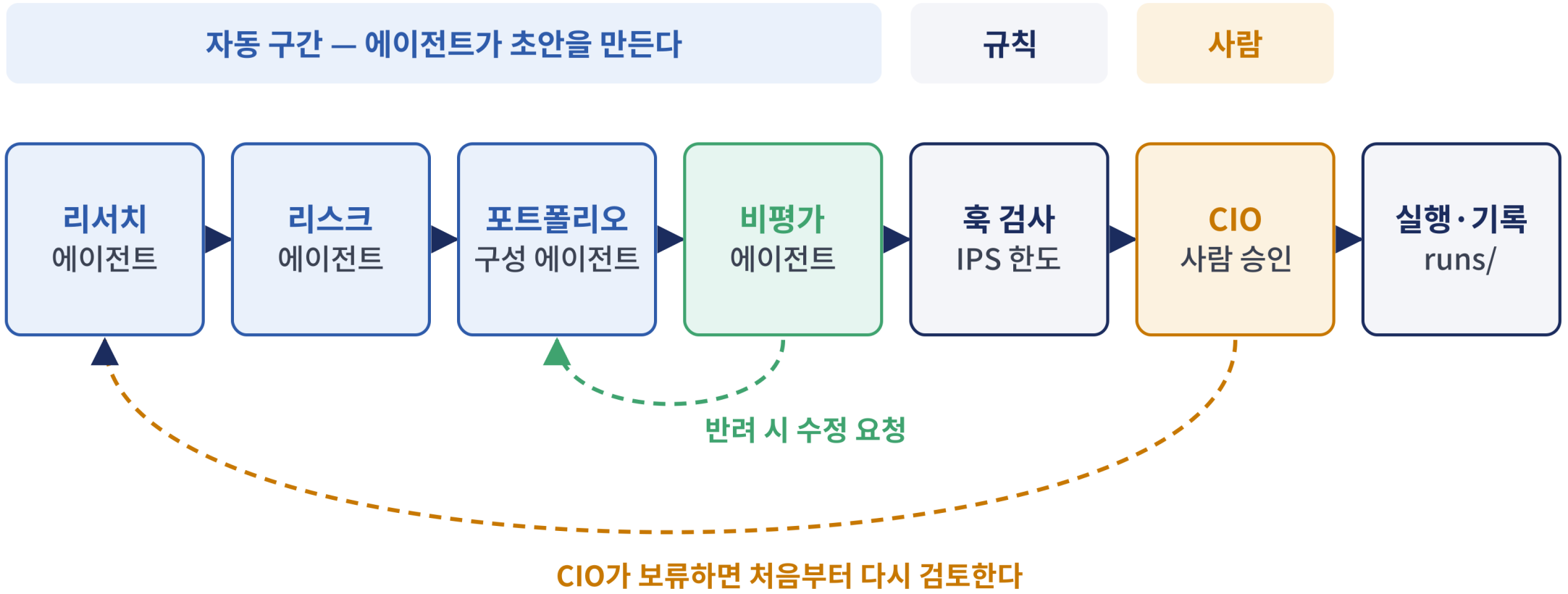
- 챗봇이 유족 할인 운임을 탑승 후에도 신청할 수 있다고 안내했다
- 실제 규정과 달랐고, 심판소는 회사에 차액 배상을 명했다

운용 시스템에 주는 교훈

- 에이전트의 출력은 회사의 의사 표시로 본다
- 대외 행동 전에는 사람 관문과 기록이 필요하다

사람 관문을 넣어 L3 자율 운용을 설계한다

에이전트가 초안을 만들고, 규칙이 걸러내고, 사람이 결재한다



관문 셋이 서로 다른 종류의 오류를 잡는다

판단 오류와 규칙 위반과 책임 소재를 나눠서 막는다

비평가 에이전트

판단 오류

근거가 약하거나 논리가 비약한 제안을 반려한다. 반대 의견을 반드시 적는다.

훅 검사

규칙 위반

IPS 한도나 원본 데이터 수정 같은 금지선을 코드로 막는다. 모델의 판단에 맡기지 않는다.

CIO 승인

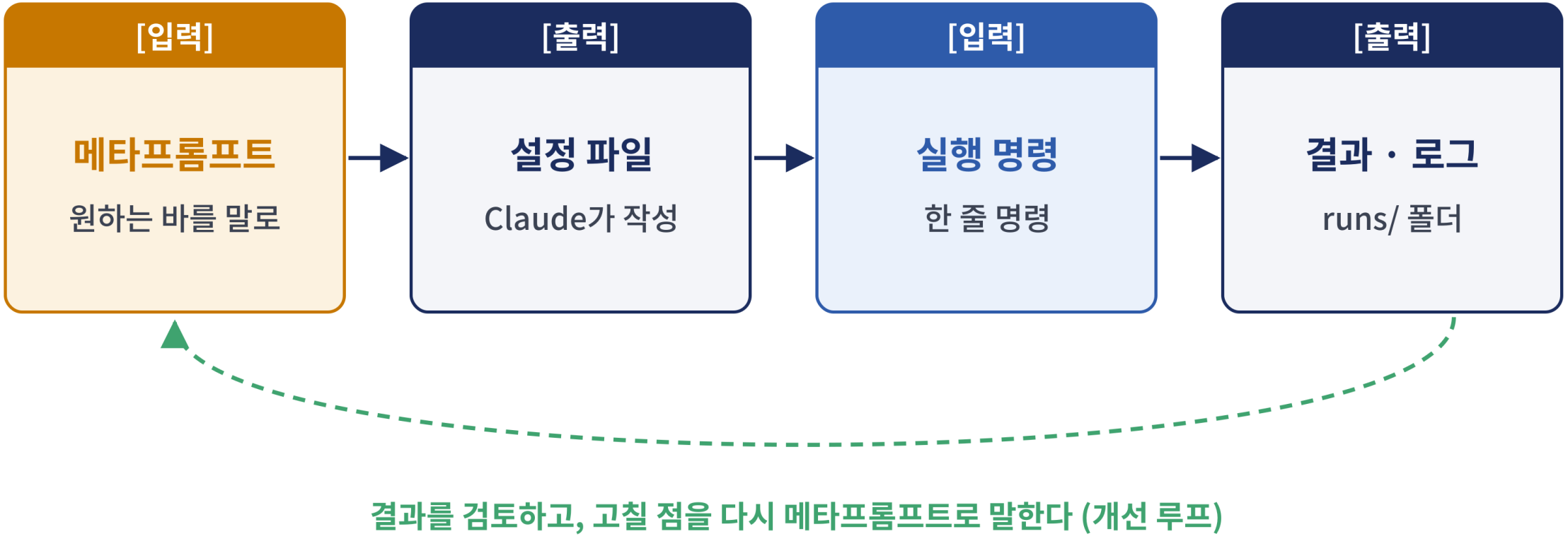
책임 소재

최종 결정과 책임은 사람이 진다. 플랜 모드와 권한 승인으로 실행 전에 멈춘다.

자동화할수록 관문의 위치를 먼저 정해야 한다

설정 파일은 직접 쓰지 않고 Claude가 쓰게 한다

메타프롬프트(metaprompt)는 프롬프트와 설정을 만들게 하는 프롬프트다



메타프롬프트 한 번으로 서브에이전트 셋을 만든다

Claude Code 입력창에 붙여 넣고 결과 파일만 검토한다

[입력] 프롬프트 · Claude Code

.claude/agents/ 에 서브에이전트 셋을 만들어 줘.

1) research: 시황과 새 논문을 읽고 요약한다
2) risk: src/risk.py 로 위험을 계산하고
   IPS 한도 위반을 판정한다. 매매 의견은 금지
3) critic: 운용안의 약점과 반대 의견을 쓴다.
   파일 쓰기 도구는 주지 마

각 파일에 name, description, tools 를 넣고
도구는 역할에 필요한 최소한만 줘.
다 만들면 각자의 도구를 표로 보여 줘.

[출력] 결과 · 생성된 파일

.claude/agents/research.md
  tools: Read, WebSearch, Write
.claude/agents/risk.md
  tools: Read, Write, Bash
.claude/agents/critic.md
  tools: Read, Glob, Grep

이름	파일 쓰기
research	가능
risk	가능
critic	불가

금지선은 메타프롬프트로 훅을 만들어 지킨다

원본 데이터 수정을 막는 가드레일을 예로 든다

[입력] 프롬프트 · Claude Code

data/ 폴더의 파일을 고치거나 지우려는 시도를
항상 막는 훅을 만들어 줘.

- .claude/settings.json 의 PreToolUse 에 등록
- 검사 스크립트는 guards/no_raw_edit.py
- 막을 때는 이유를 한 줄로 알려 줘

만든 뒤 data/ 파일을 고쳐 보는 시험을
직접 해 보고 막히는지 결과를 보여 줘.

[출력] 결과 · 시험 실행

훅 등록: PreToolUse
(Edit|Write|Bash)

시험 1: data/prices.csv 수정
→ 차단 (종료 코드 2)
→ data/ 원본은 읽기 전용입니다

시험 2: runs/.../note.md 작성
→ 허용

헤드리스 실행으로 같은 절차를 매일 돌린다

claude -p 는 대화창 없이 한 번 실행하고 끝난다

[입력] 명령어 · 터미널에 입력

```
claude -p "오늘 운용 사이클을 돌려 runs/ 에 기록해"
```

[출력] 결과 · runs/2026-10-26_01/

```
research_note.md    시황 요약
risk_report.json    breach: false
proposal.md         비중 조정안
critic_review.md    반대 의견 2건
ips_check.md        통과
status.txt          CIO 승인 대기
```

확인할 것

- 실행마다 기록 폴더가 새로 생긴다
- 마지막 상태가 승인 대기다
- 실제 매매는 승인 뒤에만 한다

모든 실행은 다시 재현할 수 있게 기록한다

운용 결정의 근거를 나중에 설명할 수 있어야 한다

기록 항목	왜 필요한가
run_id	한 번의 실행 전체를 묶는 열쇠다
입력 데이터 경로와 날짜	같은 입력으로 다시 돌릴 수 있다
모델 이름과 버전	모델이 바뀌면 결과도 바뀐다
프롬프트와 설정 파일 사본	어떤 지시로 나온 결과인지 남긴다
사람의 승인 기록	누가 무엇을 결정했는지 밝힌다

기록은 훅의 Stop 시점에 자동으로 남기게 하면 빠뜨리지 않는다

운용 자동화는 매번 성공하는지로 평가한다

pass@k와 pass^k(τ-bench, 2024)의 차이를 본다. 시행은 서로 독립이라고 둔다

$$\underbrace{\text{pass@}k = 1 - (1-p)^k}_{k\text{번 중 한 번이라도 성공}} \quad \text{VS} \quad \underbrace{\text{pass}^k = p^k}_{k\text{번 모두 성공}}$$

두 지표

- pass@k는 여러 번 중 한 번만 맞으면 된다
- pass^k는 k번 모두 맞아야 한다
- p는 한 번 실행했을 때의 성공률이다

숫자로 보면

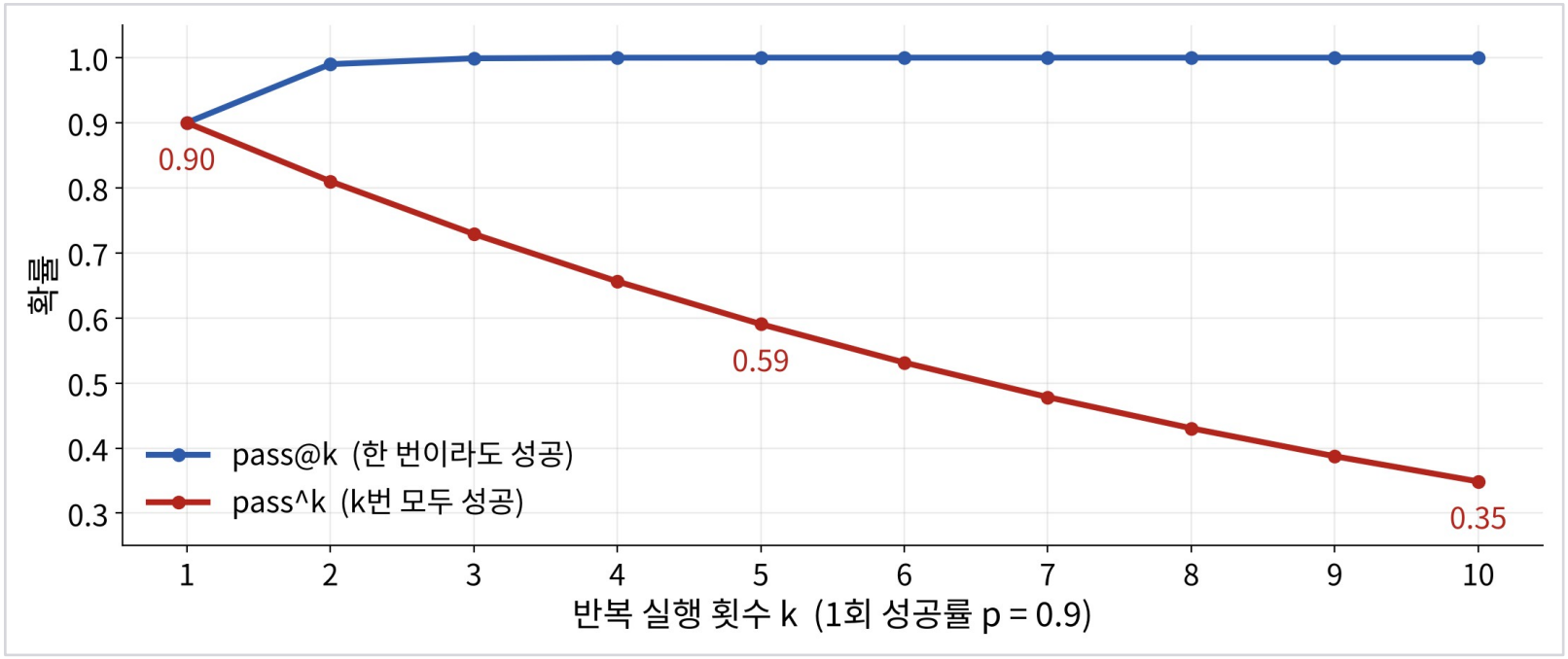
- p = 0.9여도 다섯 번 연속 성공은 0.59다
- 열 번 연속이면 0.35로 떨어진다
- 운용 자동화는 pass^k로 평가한다

한 번 잘 되는 것과 늘 잘 되는 것은 다르다

1회 성공률 p = 0.9일 때 반복 횟수에 따른 두 지표

읽는 법

- 파랑: pass@k
- 빨강: pass^k
- 운용은 빨강을 본다
- 골든셋으로 반복 측정



설계 판단을 연습하는 네 가지 질문

정답보다 근거를 말하는 데 초점을 둔다

Q1	리스크 에이전트와 비평가 에이전트를 같은 모델과 같은 지시로 만들면 무엇이 문제인가?	상관과 다수결
Q2	IPS 한도 점검을 비평가 에이전트에 맡기지 않고 훅으로 두는 이유는 무엇인가?	훅과 판단
Q3	에이전트를 5개에서 10개로 늘리자는 제안이 왔다. 무엇을 먼저 확인하겠는가?	비용과 효과
Q4	pass^5가 0.59인 운용 사이클을 L3로 운영해도 되는가? 관문은 어디에 두겠는가?	신뢰도와 관문

오늘 배운 것을 세 문장으로 정리한다

다음 시간에는 Self-Driving Portfolio 구조에 이 개념을 대입한다

1

에이전트는 모델에 도구와 루프와 기억을 붙인 것이다

모델은 컨텍스트 안의 것만 보고, 기억은 파일에 둔다. 도구 실행과 권한은 하네스가 맡는다.

2

멀티에이전트는 다양성이 있어야 검증이 된다

역할을 나누고 판단의 상관을 낮춰야 다수결과 상호 검토가 실제로 오류를 잡는다.

3

규칙은 파일로, 금지선은 훅으로, 결정은 사람이 한다

CLAUDE.md, 서브에이전트, 스킬, 훅을 메타프롬프트로 만들고 사람은 관문에서 결재한다.

다음: Ang 외(2026) Self-Driving Portfolio의 에이전트 구조를 해부한다